

```
constexpr bool empty() const;
```

³ *Returns:* **true** if the set contains no elements; otherwise, **false**.

17.10.(4 + a).11.4 Modifiers

[support.contract.control.semset.mod]

17.10.(4+a).11.4 Modifiers

[**support.contract.control.semset.mod**]

```
constexpr void clear();
```

¹ *Postconditions:* `empty()` is `true`.

```
constexpr void insert(evaluation_semantic s);
```

2 *Postconditions:* contains(s) is true.

```
constexpr void insert(initializer_list<evaluation_semantic> ilist);
```

³ *Postconditions:* For each element `s` of `ilist`, `contains(s)` is `true`.

```
constexpr void remove(evaluation_semantic s);
```

4. *Postconditions:* contains(s) is false.

17.10.(4 + a).11.5 Set operations

[support.contract.control.semset.setops]

17.10.(4+a).11.5 Set operations

[support.contract.control.semset.setops]

```
constexpr evaluation_semantic_set& operator=(const evaluation_semantic_set& rhs);
```

¹ *Postconditions:* For each enumerator `s` of `evaluation_semantic`, `contains(s)` is `true` if and only if, immediately before the call, both `contains(s)` and `rhs.contains(s)` were `true`.

2 *Returns: *this.*

```
constexpr evaluation_semantic_set& operator|=(const evaluation_semantic_set& rhs);
```

3 *Postconditions:* For each enumerator `s` of `evaluation_semantic`, `contains(s)` is `true` if and only if, immediately before the call, `contains(s)` or `rhs.contains(s)` was `true`.

4 *Returns: *this.*

```
constexpr evaluation_semantic_set operator~() const;
```

5 *Returns:* An `evaluation_semantic_set` `r` such that, for each enumerator `s` of `evaluation_semantic`, `r.contains(s)` is `true` if and only if `contains(s)` is `false`.

[illegible]

⁶ *Returns:* `evaluation_semantic_set(lhs) &= rhs.`

```
friend constexpr evaluation_semantic_set operator|(const evaluation_semantic_set& lhs,  
                                                    const evaluation_semantic_set& rhs);
```

⁷ *Returns:* `evaluation_semantic_set(lhs) |= rhs.`